

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 11 – Solutions

Instructions: You do not need to typeset your solutions; however, your submission must be a single PDF file. The assignment is due by **8:59pm on Wednesday, April 29**. Remember that you may submit as many times as you wish before the deadline (we'll only grade the last one), but late submissions will receive no credit – so submit early, submit often!

This assignment consists of problems numbered 1-5; be sure to look at the second page.

(1) A common toy for young children just learning how to spell is a set of magnetic letters on the refrigerator. (See picture at right.)



An obvious game is to spell words (allowing repeats) and try to use up all of the letters on the refrigerator. Of course, the vocabulary of young children is limited – there is only a small number of words that they actually know how to spell, and so it might not be possible to use all of the letters.

Using a reduction from 3-D Matching, prove that the problem of using up all of the refrigerator magnets is NP-complete. (Remember: This means you are taking an arbitrary instance of 3-D Matching and turning it into a Refrigerator Magnets problem.)

Very straightforward: Each object in each of the three sets of 3-D Matching becomes a unique “letter” in Refrigerator Magnets. Each allowed triple from 3DM is then a “word” in RM.

If the RM problem has a solution that uses all of the letters, then the 3DM triples corresponding to the RM words used will be a solution to 3DM, and the reverse is also true: the triples in any valid 3DM solution will be a set of words that solves RM.

Thus, $3DM \leq_P RM$, and since 3DM is NP-complete, so is RM.

(2) Two business partners want to go their separate ways. Their business has a collection of n assets; asset i is worth d_i dollars. They would like to split the assets evenly between them. Prove (using a reduction from any NP-complete problem from lecture that you like) that this task is NP-complete.

The easiest reduction is from Subset Sum. (Knapsack would be similar.)

Let $v_1, v_2, \dots, v_n$ be the values from Subset Sum, and T be the target value. Let V be the sum of all of the v_i . Create n assets in the Asset Split problem such that $d_i = v_i$. Then add two more assets: $d_x = V + T$ and $d_y = 2V - T$. Observe that the total value of all assets is now $4V$, and thus the goal is to assign $2V$ of value to each partner.

No valid solution to this AS problem can assign both of d_x and d_y to one partner, as this would give him a total value of $3V$. Thus if the AS problem has a solution, d_x goes to one partner and d_y goes to the other. Therefore, AS has a solution if and only if there is a set of the $d_1 \dots d_n$ that adds up to exactly T ; these assets go to the partner with d_y . Obviously this division of the $d_1 \dots d_n$ also solves SS, and a solution to SS will immediately solve AS.

Thus, $SS \leq_p AS$, and AS is NP-complete.

(3) Remote controlled (RC) model airplanes are operated by a pilot on the ground using a small radio-frequency transmitter that has the controls for the airplane's throttle, rudder, ailerons, etc. The frequencies used for these are rigidly controlled – if two controllers are trying to use the same frequency at the same time, their signals will interfere and likely both planes will crash.

Pilots at a field will request flight times, each of them with a start time and an end time. Give a greedy algorithm that, given inputs of n requested flight times, will determine the minimum number of frequencies needed to ensure that there are no overlaps.

This is functionally equivalent to a previous HW problem.

Most efficient is to make two copies of the request list, one sorted by end time and one by start time. Find the request r with the earliest end time. Find all requests with start times earlier than that; these necessarily overlap with r . Count how many overlapping requests there were, and then remove all of those requests from both lists. Repeat this process, keeping track of the largest number of overlaps – this will be the minimum number of frequencies needed to avoid conflicts.

(4) Given an array with n (not necessarily distinct) values, the goal is to find the maximum difference between any two elements. (This is also sometimes called the *range* of the array.) For example, given $[4.5, 10, -2, 3.14159, -7.1]$, the correct answer is 17.1.

Give a divide-and-conquer algorithm that solves this problem in $O(n)$ time. Use the Master Theorem to verify the runtime of your algorithm.

APOLOGIES: The obvious D&C algorithm is actually $O(n \log n)$. (There might also be a linear time D&C algorithm, but if so it's more complex.) It's worth noting that a simple linear search will also solve this in linear time: Just find the min and find the max, then subtract them.

To do it divide-and-conquer (which is less efficient in this case), we argue that the max difference is either the same as the max difference in the first half, the same as the max difference in the second half, or it's the max difference that crosses the center (with the largest element in one half of the list and the smallest in the opposite half).

Our algorithm thus looks like this:

```
def max_diff(L,start,end): # start and end are the first and last
    if start == end:       # indices of L that we are currently
        return 0           # considering
    mid = (start + end) // 2 # integer division
    a = max_diff(L,start,mid)
    b = max_diff(L,mid+1,end)
    minLeft =  $\infty$ ; maxLeft =  $-\infty$ 
    for i = start to mid:
        if L[i] < minLeft: minLeft = L[i]
        if L[i] > maxLeft: maxLeft = L[i]
    minRight =  $\infty$ ; maxRight =  $-\infty$ 
    for i = mid+1 to end:
        if L[i] < minRight: minRight = L[i]
        if L[i] > maxRight: maxRight = L[i]
    c = maxLeft - minRight
    d = maxRight - minLeft
    return min(a,b,c,d)
```

Since we are doing a linear amount of (non-recursive) work with each function call, the recurrence relation is $T(n) = 2T(n/2) + O(n)$. Our Master Theorem coefficients are $a = 2, b = 2, d = 1$. Since $d = \log_b a$, we're in the middle case, which gives $O(n \log n)$.

(5) In most large cities, one tends to need to ride mass transit daily. Transit passes are sold in blocks of 1, 7, and 30 days. Furthermore, the costs of these passes vary on a daily basis based upon demand, holidays, etc. Let $D[i]$, $W[i]$, $M[i]$ represent the costs of a 1-day, 7-day, or 30-day transit pass purchased on day i .

Give a linear-time dynamic programming algorithm that will find the minimum cost way to buy transit access for the next n days.

This one ended up being a little trickier than I thought. I *intended* to give a problem where you just had to look at $OPT[i - 1]$, $OPT[i - 7]$, and $OPT[i - 30]$. However, it was pointed out to me that today's travel could be covered by a weekly pass purchased during any of the previous seven days, so we need to tweak things a bit.

$OPT[i]$ will be the total cost of travel for the first i days of the trip; obviously, $OPT[0] = 0$. On any given day i , we could be traveling on a daily, weekly, or monthly pass. Daily passes were obviously bought that day, but we could have bought the others on any of a number of previous days. We don't know which is best, so we'll need to minimize across all possibilities. This gives us the following recurrence:

$$OPT[i] = \min \begin{cases} D[i] + OPT[i - 1] \\ \min_{0 \leq k \leq 6} W[i - k] + OPT[i - 1 - k] \\ \min_{0 \leq k \leq 29} M[i - k] + OPT[i - 1 - k] \end{cases}; OPT[0] = 0; OPT[i] = \infty \text{ for } i < 0$$

To clarify: If we are traveling today on a weekly pass, we might have bought it today or on any of the six previous days. In each case, the optimal total cost now is based on the optimal total on the day before we bought the pass. (Similarly for a monthly pass.) We set OPT to infinity for negative values to effectively remove them from consideration – this is notationally more convenient than making exceptions for k , but when implementing, we probably wouldn't do it quite this way.

For each day, we need to calculate up to 38 different values. But, 38 is a constant, and so we are doing a constant amount of work each day. Thus, the overall algorithm is linear time.